Information Modeling Using EXPRESS — Advanced Concepts

NB. NOTES ARE ON EVEN NUMBERED (LEFT HAND) PAGES AND THE PRESENTATION ON ODD NUMBERED (RIGHT HAND) PAGES.

Extensibles

EXPRESS Edition 2 (2004) introduced EXTENSIBLE types. These are SELECT and ENUMERATION types whose item lists can be extended. This provides increased flexiblity and reuse in modeling.

In some sense, EXTENSIBLE types are analagous to SUBTYPEs, except that subtyping reduces the domain while extending increases the domain.

Extensibles

- SELECT specifies a list of things to choose from
- ENUMERATION specifies a list of items (names)
- It is possible to extend these lists

```
TYPE approval = EXTENSIBLE ENUMERATION OF
     (approved, rejected); END_TYPE;

TYPE your_approval = ENUMERATION BASED_ON
     approval WITH (cancelled); END_TYPE;

TYPE my_approval = EXTENSIBLE ENUMERATION
     BASED_ON approval WITH (pending);
END_TYPE;
```

Relationships

If one entity serves as an attribute of another entity, then the two entities are related, or more precisely there is a relationship between the two entities.

A `tt "door"` must have at least two `tt "hinge"`s.

A `tt "door"` must have a `tt "handle"`.

Relationships

When an attribute of an entity is of type entity, then a relationship is established between the two entity types.

```
ENTITY door;
  hinges : SET [2:?] OF hinge;
  knob   : handle;
  ...
END_ENTITY;

ENTITY hinge;
  id : name;
  ...
END_ENTITY;

ENTITY handle;
 ...
END_ENTITY;
```

Cardinality

Cardinality is how many of one thing is needed/used by another thing. (‘Cardinality’ is related to ‘cardinal number’ — a number used for counting).

The cardinality of the ‘attribute’ entity with respect to the ‘owning’ entity is defined in the owning entity.

By default the cardinality of the owning entity with respect to the attribute entity is zero to many.

The attribute entity can change this via an INVERSE attribute.

Cardinality

- Cardinality constraint from the referring entity to the referenced entity is specified via the attribute definition (e.g OPTIONAL, aggregation).
- Default cardinality constraint from the referenced to the referring entity is zero to many `[0:?]`.
- `INVERSE` attributes can be used to specify other cardinalities.

Cardinality (cont)

In the last model a `tt "door"` required one `tt "handle"` but from the point of view of a `tt "handle"`, any number could be used on a `tt "door"` (or yhis could be restated as `tt "handles"` know nothing about `tt "doors"`).

In this model a `tt "handle"` can be used on either one or no doors.

Cardinality (cont)

```
ENTITY door;
  hinges : SET [2:?] OF hinge;
  knob   : handle;
  ...
END_ENTITY;

ENTITY handle;
 ...
INVERSE
  knob_for : SET [0:1] OF door FOR knob;
END_ENTITY;
```

Here, the INVERSE clause states that a handle is 'used' by zero or one doors.

ABSTRACT ENTITY

This was introduced in EXPRESS Edition 2.

It has to be instantiated (e.g., appear in a Part 21 file) via a SUBTYPE in which the specific type of each attribute is specified.

The model here says that a `tt "general_approval"` has a `tt "status"` which is of type `tt "approval_status"` (whatever that is), and `tt "approved_items"` (which is a SET of (unnamed) ENTITY). An instantiable SUBTYPE would have to replace `tt "GENERIC_ENTITY"` by a named entity, say `tt "plan"` (whatever that might be), to have a set of approved `tt "plan"`s.

ABSTRACT ENTITY

An ENTITY that can only be instantiated via its SUBTYPEs.

```
ENTITY general_approval ABSTRACT;
  approved_items : SET OF GENERIC_ENTITY;
  status         : aproval_status;
END_ENTITY;
```

- GENERIC_ENTITY stands for any ENTITY
- All attributes must have specific types before instantiation is possible.
- An attribute type can be redeclared to a more specific kind in a SUBTYPE

Example person Entity

We are going to use this as the basis for the next few slides.

Example person ENTITY

```
ENTITY person;
  first_name : STRING;
  last_name  : STRING;
  nickname   : OPTIONAL STRING;
  ss_no      : INTEGER;
  gender     : sex;
  spouse     : OPTIONAL person;
  children   : SET [0:?] OF person;
UNIQUE
  un1 : ss_no;
WHERE
  w1 : (EXISTS(spouse) AND
        gender <> spouse.gender)
       OR NOT EXISTS(spouse);
END_ENTITY;
```

IS-A Relationship

The database world talks about IS-A relationships, for instance THIS IS-A THAT, or A CAR IS-A (kind of) VEHICLE.

In EXPRESS a SUBTYPE IS-A (more special kind of its) SUPERTYPE(s).

Conversely a SUPERTYPE IS-A (more general kind of its) SUBTYPE(s).

IS-A Relationship

- EXPRESS supports the IS-A relationship via subtyping.
- Entities S1, S2, … can be declared to be SUBTYPES of entity E. This also effectively declares E to be a SUPERTYPE of S1, S2, etc.

  That is, S1 is-a E, S2 is-a E, etc. Also, E may-be an S1, E may-be an S2.
- An entity may be both a SUB- and a SUPERTYPE.
- An entity may be a SUBTYPE of more than one entity.
- SUPER/SUBTYPING may be used for many purposes.

Inheritance

A SUBTYPE is a special kind of its SUPERTPE(s). There are fewer instances of a SUBTYPE than of its SUPERTYPE. For example, there are fewer CARS than there are VEHICLES.

A SUBTYPE inherits all the attributes and constraints of its SUPERTYPE(s).

A SUBTYPE can have additional attributes and constraints.

This revised `tt "person"` model eliminates the original WHERE rule about spouses being of opposite sex. We can also talk about a `tt "person"` without having to identify the person's gender.

Inheritance

A SUBTYPE inherits all the attributes and constraints of its SUPERTYPE(s).

```
ENTITY person;
  first_name : STRING;
  last_name  : STRING;
  ss_no      : INTEGER;
  children   : SET [0:?] OF person;
UNIQUE
  un1 : ss_no;
END_ENTITY;

ENTITY male
  SUBTYPE OF (person);
  wife : OPTIONAL female;
END_ENTITY;

ENTITY female
  SUBTYPE OF (person);
  husband : OPTIONAL male;
END_ENTITY;
```

SUBTYPE instance constraints

We can use this model to talk about a

- A person
- A person who is an employee
- A person who is a student
- A person who is an employee and who is also a student

SUBTYPE instance constraints

- In general, an instance of a Supertype may involve instances of zero or more of its Subtypes.

```
ENTITY person;
  ...
END_ENTITY;

ENTITY employee
  SUBTYPE OF person;
  ...
END_ENTITY;

ENTITY student
  SUBTYPE OF person;
  ...
END_ENTITY;
```

- If this is not the required behaviour, then the ‘instance set’ can be constrained.

SUBTYPE_CONSTRAINT

The SUBTYPE_CONSTRAINT construct was introduced in EXPRESS Edition 2.

In Edition 1 the constraint specification was lexically embedded in the definition of the Supertpye entity. If a new subtytpe was introduced in a different Schema that imported the Supertype there was no convenient method, apart from changing the original Supertype definition, of constraining the use of the new Subtype.

Multiple SUBTYPE_CONSTRAINTs can be applied to a Supertype. The constraints are additive. (In EXPRESS you cannot eliminate a constraint).

SUBTYPE_CONSTRAINT

```
SUBTYPE_CONSTRAINT sc FOR ent;
-- constraints
END_SUBTYPE_CONSTRAINT;
```

specifies SUBTYPE constraints for ENTITY ent.

Several SUBTYPE_CONSTRAINTs can be specified for any given ENTITY. The constraints are additive.

SUBTYPE Constraint Summary

In general, an instance of a Supertype can involve any of its Subtypes.

The constraints are used to eliminate certain combinations of Subtypes.

The particulars are described later.

SUBTYPE Constraint Summary

- No constraints: An instance of the Supertype involves zero or more Subtype instances.
- ABSTRACT SUPERTYPE: An instance of the Supertype must involve one or more Subtype instances.
- TOTAL_OVER(x,y) means that every instance of the Supertype must involve an instance of at least one of the listed Subtypes.
- ONEOF(x,y,z) means that one and only *one of* the listed Subtypes can be instanced with an instance of the Supertype.
- (x ANDOR y) means that an instance of the Supertype may be accompanied by instances of the Subtypes x *and/or* y (the default condition).
- (x AND y) means that an instance of the Supertype may be accompanied by instances of the Subtypes x *and* y.

ABSTRACT SUPERTYPE

An ABSTRACT SUPERTYPE can only be instantiated in conjunction with non-ABSTRACT subtype(s).

ABSTRACT SUPERTYPE

- An entity does not have to declare itself to be a SUPERTYPE. It is a SUPERTYPE if it is mentioned by a SUBTYPE.
- In some cases, a Supertype is not to be instantiated without one of its Subtypes. The entity can be constrained to be an ABSTRACT SUPERTYPE.

```
ENTITY mammal
  ...
END_ENTITY;

SUBTYPE_CONSTRAINT sc_abs FOR mammal;
  ABSTRACT SUPERTYPE;
END_SUBTYPE_CONSTRAINT;

ENTITY dog
  SUBTYPE OF mammal;
  ...
```

```
END_ENTITY;
```

TOTAL_OVER

This was introduced in Edition 2 (I have failed to find any use for it).

It means (I think) that the listed Subtypes completely cover the domain of the Supertype. Further, every instance of the Supertype that includes Subtype instances must include an instance of one of the listed subtypes.

TOTAL_OVER

```
ENTITY person;
...
END_ENTITY;

SUBTYPE_CONSTRAINT adultchild FOR person;
  TOTAL_OVER(adult,child);
END_SUBTYPE_CONSTRAINT;

ENTITY child SUBTYPE OF (person);
END_ENTITY;

ENTITY adult SUBTYPE OF (person);
END_ENTITY;

ENTITY student SUBTYPE OF (person);
END_ENTITY;
```

Every person is either a child or an adult. A student is also either a child or an adult.

ONEOF

A ONEOF constraint means that one and only ONE OF the listed subtypes can be used in an instance of the Supertype.

Here the constraint is that a person cannot be simultaneously a male and a female. Note that if the constraint was not there (as in the earlier model) it would mean that the model catered for hermaphrodites, which would introduce a new set of problems.

ONEOF

```
ENTITY person;
  first_name : STRING;
```

```
  last_name  : STRING;
  ss_no      : INTEGER;
  children   : SET [0:?] OF person;
UNIQUE
  un1 : ss_no;
END_ENTITY;

SUBTYPE_CONSTRAINT mf FOR person;
  ONEOF(male, female);
END_SUBTYPE_CONSTRAINT;

ENTITY male
  SUBTYPE OF (person);
  wife : OPTIONAL female;
END_ENTITY;

ENTITY female
  SUBTYPE OF (person);
  husband : OPTIONAL male;
END_ENTITY;
```

ANDOR

P ANDOR Q means that the following combinations of subtypes are allowed:

- P only
- Q only
- P and Q together.

That is P and/or Q are allowed.

The unconstrained relationship between Subtypes (the default) is ANDOR.

In the example model the constraint might as well not be there.

ANDOR

```
ENTITY person;
  first_name : STRING;
  last_name  : STRING;
  ss_no      : INTEGER;
  children   : SET [0:?] OF person;
UNIQUE
  un1 : ss_no;
END_ENTITY;

SUBTYPE_CONSTRAINT es FOR person;
```

```
  employee ANDOR student;
END_SUBTYPE_CONSTRAINT;

ENTITY employee
  SUBTYPE OF (person);
  salary : REAL;
END_ENTITY;

ENTITY student
  SUBTYPE OF (person);
  fees : REAL;
END_ENTITY;
```

AND

P AND Q means that if there is an instance of P it must be accompanied by an instance of Q, and vice-versa — either both or none.

The example shows that the constraints may be complex (logical) expressions.

Unconstrained there are 15 possible combinations (from Person to a male, female, citizen, alien person).

With the given constraints there are only 5 (Person, (fe)male citizen, (fe)male alien).

AND

```
ENTITY person;
  ...
END_ENTITY;

SUBTYPE_CONSTRAINT mf_and_ca FOR person;
  ONEOF(male, female) AND
  ONEOF(citizen, alien);
END_SUBTYPE_CONSTRAINT;

ENTITY male SUBTYPE OF (person);
 ...
END_ENTITY;

ENTITY female SUBTYPE OF (person);
 ...
END_ENTITY;

ENTITY citizen SUBTYPE OF (person);
END_ENTITY;

ENTITY alien SUBTYPE OF (person);
```

```
END_ENTITY;
```

SUBTYPEs

Much of this list has already been touched on. The first item is part of the ‘meaning’ of SUBTYPE.

The following example includes examples of the last 3 elements in the list.

SUBTYPEs

- A Subtype is a specialisation of its Supertype(s).
- New attributes may be added.
- New constraints may be added.
- Attributes may be ‘retyped’ (i.e their domains may be specialised in a compatible manner).

SUBTYPEs (cont)

A simple example showing:

- Attribute redeclaration
- Adding attribute(s)
- Adding constraint(s)

SUBTYPEs (cont)

```
ENTITY circle;
  radius : NUMBER;
  center : point;
END_ENTITY;

ENTITY specialised_circle
  SUBTYPE OF (circle);
  SELF\circle.radius : REAL;    -- retyped
  shade  : colour; -- additional attribute
WHERE
  SELF\circle.radius > 3.0; -- add constraint
END_ENTITY;
```

QUERY Expression

Now we are getting away from structural modeling.

QUERY Expression

The query expression evaluates a logical expression against each element of an aggregation, returning an aggregation of all the elements for which the logical expression is TRUE.

The syntax is roughly:

```
QUERY( temp <* agg | lexp)
```

where `temp` is the name of a temporary variable, `agg` is the aggregation, and `lexp` is the logical expression.

For example, assuming that a person's attributes included the age of the person,

```
QUERY(t <* persons | t.age >= 21)
```

would return all the people whose age was 21 or greater.

QUERY (cont)

You can't actually write this function in EXPRESS (if you could the QUERY expression would probably not have been invented), as there is no LOGICAL_EXPRESSION type in the language.

An example of its use follows.

QUERY (cont)

The effect of QUERY is similar to the pseudo-function below.

```
FUNCTION q(agg  : AGGREGATE OF GENERIC;
           lexp : LOGICAL_EXPRESSION;)
          : AGGREGATE OF GENERIC;
LOCAL
  result : AGGREGATE OF GENERIC := [];
END_LOCAL;
  REPEAT i := 1 TO SIZEOF(agg);
    IF (lexp = TRUE) THEN
      result := result + agg[i];
    END_IF;
  END_REPEAT;
RETURN(result);
END_FUNCTION;
```

RULE

A WHERE rule in an ENTITY applies to each and every instance of the ENTITY.

A RULE is a constraint that can be applied to either some instances of a particular ENTITY or to combinations of instances of different ENTITY (types).

Given a database of instances, each RULE is applied to every applicable instance in the database to determine if the instance conforms to the constraint.

EXPRESS assumes that every (ENTITY) instance has a unique identifier, although it does not specify what that might be. You could have two (or more) instances of a `tt "point"` with the same coordinate values but they are still distinguisable fronm each other in the storage system.

RULE

- Local constraints (WHERE, UNIQUE, INVERSE) are applied to each and every instance of the entity.
- Global constraints (RULEs) are applied between entities or across a subset of entity instances.

The following rule states that there shall be one and only one point at the origin in the objectbase.

```
RULE unique_origin FOR (point);
LOCAL
  origin : BAG OF point;
END_LOCAL;
  origin := QUERY(temp <* point |
                  (temp.x = 0.0) AND
                  (temp.y = 0.0) );
WHERE
  r1 : SIZEOF(origin) = 1;
END_RULE;
```

RULE (cont)

Creating a robust EXPRESS model is not necessarily easy.

Going back to the Person/male/female model it does say that wifes are females and husbands are males. It doesn't say that if Adam claims his wife to be Eve then Eve's husband must be Adam.

In some communities that might not be a problem. But, if it is in the bit of the real world that the model represents then the rather complicated RULE fixes that relationship problem.

It looks at every male and checks to see if he is his wife's husband. It also has to look at every female to see if she is her husband's wife.

The double check is needed for the cases when one of a pair claims to be single.

NOTE EXPRESS does not specify when the RULEs should be checked.

RULE (cont)

This RULE states that husbands and wives must be married to each other.

```
RULE married FOR (male,female);
  LOCAL
    ok1, ok2 : BOOLEAN := TRUE;
  END_LOCAL;
  IF (EXISTS(male.wife) AND
      male :<>: male.wife.husband) THEN
    ok1 := FALSE;
  END_IF;
  IF (EXISTS(female.husband) AND
      female :<>: female.husband.wife) THEN
    ok2 := FALSE;
  END_IF;
WHERE
  r1 : ok1;
  r2 : ok2;
END_RULE;
```

SCHEMA Interfacing

An EXPRESS model typically consists of several SCHEMAs, each dealing with a distinguishable subtopic.

Anything in a SCHEMA can be utilised by any other SCHEMA — you can't hide anything — but you have to specify what you want.

The contents of a SCHEMA are ENTITY, TYPE, RULE, SUBTYPE_CONSTRAINT, FUNCTION, PROCEDURE and CONSTANT declarations, each of which has a name.

Within a SCHEMA all the names must be unique.

When importing something from another SCHEMA it may be necessary to rename it if its name is already declared, or it may convey the semantics better if it was called by a different name.

SCHEMA Interfacing

- Definitions within a Schema are potentially available to all Schemas.
- Definitions have to be 'imported' from the original Schema into the 'current' Schema.
- An imported definition implicitly imports all the necessary definitions to complete the definition.

EXPRESS syntax is roughly

```
import FROM schema_ref (def1 AS newname1,
                        def2 AS newname2);
```

USE Import

Only ENTITYs and TYPEs can be USEd into a SCHEMA.

A USEd ENTITY is a *first class* item. That means that in the object base instances do not need to be referenced by other instances.

It is as though the ENTITY had been declared in the using schema. Following from this, USEs can be chained.

Any items needed to complete the definitions of USEd items are implicitly REFERENCEd into the schema.

USE Import

- Only ENTITYs and TYPEs can be imported via a USE statement.
- USEd ENTITYs are 'first class' items (i.e they can be independently instantiated).
- The 'stuff' required to complete the definitions of an imported item are implicitly REFERENCEd into the schema.
- If no list is given, *all* ENTITYs and TYPEs in the SCHEMA are imported.
- USEs can be chained.

NOTE

If `fc` is a first-class entity, then the statement

```
SIZEOF(USEROF(fc)) >= 0;
```

holds.

USE (cont)

Here is a demonstration 2 schema model where an enity declared in one schema is USEd by the other.

Following this is an equivalent model expanding out the USE.

USE (cont)

```
SCHEMA source;
  ENTITY e1;
    attr : t1;
  END_ENTITY;

  TYPE t1 = REAL; END_TYPE;
END_SCHEMA;

SCHEMA using;
  USE FROM source (e1);

  ENTITY e2;
    attr : SET OF e1;
  END_ENTITY;
END_SCHEMA;
```

gives effectively:

USE (cont)

In the expanded model, SCHEMA `tt "source"` is unchanged.

SCHEMA `tt "using"` is changed with the USE being replaced by:

- ENTITY `tt "e1"` is declared
- TYPE `tt "t1"` is REFERENCED from SCHEMA `tt "source"` to provide for the `tt "attr"` attribute of `tt "e1"` (which was originally implicitly referenced).

USE (cont)

```
SCHEMA source;
  ENTITY e1;
    attr : t1;
  END_ENTITY;

  TYPE t1 = REAL; END_TYPE;
END_SCHEMA;

SCHEMA using;
  REFERENCE FROM source (t1);

  ENTITY e1;
    attr : t1;
  END_ENTITY;
```

```
  ENTITY e2;
    attr : SET OF e1;
  END_ENTITY;
END_SCHEMA;
```

RERERENCE Import

Effectively, any kind of item can be REFERENCEd — ENTITY, TYPE, FUNCTION …

REFERENCEd ENTITYs are second class items (only instances that are used as attribute(s) in other ENTITYs are allowed).

Items required to complete declarations are implicitly REFERENCEd, but there is no chaining.

A REFERENCE with just the SCHEMA name references everything in the SCHEMA.

If an item is both USEd and REFERENCEd, it is treated as being USEd.

REFERENCE Import

- Any kind of item can be imported via a REFERENCE statement.
- A REFERENCE is necessary to resolve references (links) to declarations in other schemas.
- REFERENCEDd items are ‘second class’ items (i.e they can not be independently instantiated).
- The ‘stuff’ required to complete the definitions of an imported entity are implicitly REFERENCEd into the schema.

NOTE

If `sc` is a second-class entity, then the statement

```
SIZEOF(USEROF(sc)) >= 1;
```

holds.

REFERENCE (cont)

This model is the same as the earlier one except that USE is replaced by REFERENCE.

An expanded version follows.

REFERENCE (cont)

```
SCHEMA source;
  ENTITY e1;
```

```
    attr : t1;
  END_ENTITY;

  TYPE t1 = REAL; END_TYPE;
END_SCHEMA;

SCHEMA referencing;
  REFERENCE FROM source (e1);

  ENTITY e2;
    attr : SET OF e1;
  END_ENTITY;
END_SCHEMA;
```

gives effectively:

REFERENCE (cont)

In the expanded model, SCHEMA `tt "source"` is unchanged.

SCHEMA `tt "using"` is changed with the REFERENCE list expanded to include the TYPE `tt "t1"` (which was originally implicitly referenced).

REFERENCE (cont)

```
SCHEMA source;
  ENTITY e1;
    attr : t1;
  END_ENTITY;

  TYPE t1 = REAL; END_TYPE;
END_SCHEMA;

SCHEMA referencing;
  REFERENCE FROM source (e1, t1);

  ENTITY e2;
    attr : SET OF e1;
  END_ENTITY;
END_SCHEMA;
```

SCHEMA Interfacing

A SCHEMA can extend and/or constrain a model in another SCHEMA.

In SCHEMA `tt "second"`, `tt "bbb"` (which is `tt "aaa"` under another name)and `tt "constrained"` are

first class entities. Entity `tt "original"`, which is now a SUPERTYPE of `tt "constrained"`, is second class (every instance of `tt "original"` must also be an instance of `tt "constrained"`).

Within SCHEMA `tt "first"`, entity `tt "original"` does not know it is a SUPERTYPE as `tt "first"` knows nothing about the `tt "second"` SCHEMA.

SCHEMA Interfacing

```
SCHEMA first;
  ENTITY aaa;
    -- attributes
  END_ENTITY;

  ENTITY original;
    attr : NUMBER;
  END_ENTITY;
END_SCHEMA; -- first

SCHEMA second;
  USE FROM first (aaa AS bbb);
  REFERENCE FROM first (original);

  ENTITY constrained
    SUBTYPE OF (original);
    attr : INTEGER(7);
    WHERE
      positive : attr > 0;
    END_ENTITY;
END_SCHEMA; -- second
```

EXPRESS Summary

It's a great family of languages.

EXPRESS Summary

- A powerful OO information modeling language
  - Primary form is a computer processible text language.
  - EXPRESS-G as a graphical subset.
  - EXPRESS-I as an instantiation form
  - EXPRESS-X transformation specification
- Is an ISO standard language.
- Normative STEP information models.

- Becoming widely used in the modeling communities.
- Software tools available.